



Язык программирования C++

Лекция 1

ЯЗЫК C++



“

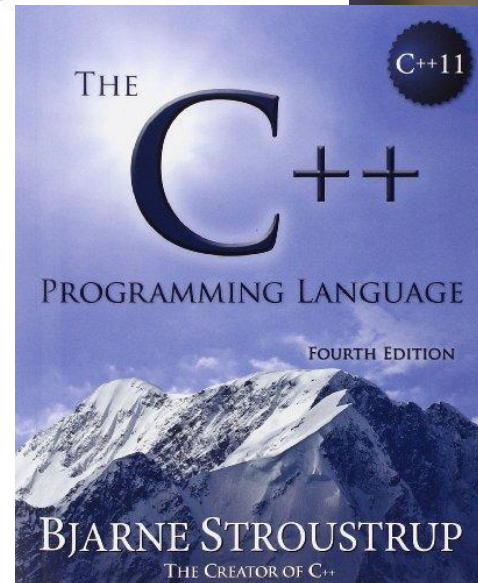
```
#include <stdio.h>
main()
{
    printf("hello, world\n");
}
```

Dennis M. Ritchie (1941-2011)

THE C PROGRAMMING LANGUAGE

Brian W. Kernighan • Dennis M. Ritchie

PRENTICE HALL SOFTWARE SERIES



АЛФАВИТ ЯЗЫКА C++

Алфавит – это совокупность символов, используемых в языке

- строчные и прописные буквы латинского алфавита
- цифры
- знак подчеркивания
- некоторые специальные символы

Группа символов	Символы
Знаки препинания	, . ; : ? ' "
Скобки	() [] { }
Операции отношения	< > =
Арифметические операции	+ - * /
Прочие символы	% & ^ \ ~ #

ПОНЯТИЕ ЛЕКСЕМЫ

Лексема – это единица текста программы, которая воспринимается компилятором как единое целое

- идентификаторы
- ключевые слова
- знаки операций
- константы
- разделители

ESCAPE-ПОСЛЕДОВАТЕЛЬНОСТИ

`\n` – переход на
новую строку

`\t` –
горизонтальная
табуляция

`\v` –
вертикальная
табуляция

`\r` – возврат
каретки в
начало строки

`\"` – кавычки

`\'` – апостроф

`\\` – обратный
слеш

`\0` – нулевой
символ

`\a` – звуковой
сигнал

`\?` – знак
вопроса

СКАЛЯРНЫЕ ТИПЫ ДАННЫХ. СПЕЦИФИКАТОРЫ

int (целый)

float (вещественный)

double (вещественный с двойной точностью)

char (символьный)

wchar_t (расширенный символьный)

bool (логический)

short (короткий)

long (длинный)

signed (знаковый)

unsigned (беззнаковый)

[<спецификатор>] <тип_переменной> <имя_переменной>[(<начальное значение>)]

ЦЕЛЫЙ ТИП (INT)

Тип данных	Диапазон значений	Размер
short int signed short int	-32 768 ... 32 767	2 байта
unsigned short int	0... 65 535	2 байта
int signed int signed long int	-2 147 483 648 ... 2 147 483 647	4 байта
unsigned int unsigned long int	0 ... 4 294 967 295	4 байта
long long int	$-2^{63} \dots (2^{63}-1)$	8 байт
unsigned long long int	$0 \dots (2^{64}-1)$	8 байт

ТИПЫ С ПЛАВАЮЩЕЙ ТОЧКОЙ (FLOAT, DOUBLE И LONG DOUBLE)

Тип данных	Диапазон значений	Размер
float	3.4E-38 ... 3.4E+38	4 байта
double	1.7E-308... 1.7E+308	8 байт
long double	3.4E-4932 ... 3.4E+4932	10 байт

СТРУКТУРА ПРОГРАММЫ НА ЯЗЫКЕ C++

Директивы препроцессора

Объявления (описание глобальных переменных, прототипов функций, структур, классов и типов данных)

Описание самих функций

```
директивы препроцессора
объявления
тип_результата main(параметры)
{
    тело функции main
}
тип_результата F1(параметры)
{
    тело функции F1
}
тип_результата F2(параметры)
{
    тело функции F2
}
...
тип_результата FN(параметры)
{
    тело функции FN
}
```

```
void main()
{
    <тело функции>;
}
```

```
int main()
{
    <тело функции>;
    return 0;
}
```

ДИРЕКТИВЫ ПРЕПРОЦЕССОРА

- #include <имя файла>
- #include "имя файла"

```
#include <iostream>  
#include "new_module.h"
```

- #define <идентификатор> <значение>

```
#define G 9.8
```

ПОТОЧНЫЙ ВВОД И ВЫВОД ДАННЫХ

Поток – это последовательность байтов, к которым можно получить доступ

std::cin >> <переменная 1> >> <переменная 2> ... >> <переменная N>;

```
std:: cin >> x >> y >> z;
```

std::cout << <выражение 1> << <выражение 2> ... << <выражение N>;

```
std:: cout << x << y << z;
```

```
std:: cout << x << ' ' << y << ' ' << z;
```

ПОНЯТИЕ ПРОСТРАНСТВА ИМЕН

Пространство имен – это область программного кода, где гарантируется уникальность всех идентификаторов

namespace <имя пространства имен>

<имя пространства имен>::<имя программного объекта>

```
#include <iostream>
using namespace std;
void main()
{
    int x, y;
    cin >> x >> y;
    cout<< x << ' ' << y;
}
```

ФОРМАТИРОВАННЫЙ ВЫВОД. МАНИПУЛЯТОРЫ

Манипулятор – это объект, который управляет потоками ввода/вывода для форматирования передаваемой в потоки информации

oct – вывод величины в восьмеричной системе счисления (целых чисел!)

hex – вывод величины в шестнадцатеричной системе счисления (целых чисел!)

fixed – вывод чисел в формате с фиксированной точкой

setprecision(n) – вывести n знаков после десятичной точки (используется совместно с fixed)

scientific – вывод чисел в экспоненциальной форме

endl – переход на новую строку

boolalpha – вывод логических величин в текстовом виде

setw(n) – установить ширину выводимого поля в n символов

left – выравнивание по левой границе

right – выравнивание по правой границе

cout << <манипулятор> << <выражение>;

ПРИМЕР

13.58
12

```
#include <iostream>
#include <iomanip>
using namespace std;
void main()
{
    int x = 18;
    float y = 13.578;
    cout<< fixed << setprecision(2) << y << endl;
    cout<< setw(5) << hex << x;
}
```

УНАРНЫЕ ОПЕРАЦИИ

Инкремент (++) и **декремент (--)** – это операции увеличения и уменьшения значения на 1

Префиксная форма – сначала изменяется операнд, а затем его значение становится результирующим значением выражения

Постфиксная форма – значением выражения является исходное значение операнда, после чего он изменяется

```
int a = 3;  
int b = 4;  
cout << a * ++b - 11;  
Результат: 4.
```

```
int a = 3;  
int b = 4;  
cout << a * b++ - 11;  
Результат: 1.
```

УНАРНЫЕ ОПЕРАЦИИ

Операции отрицания (-, ! и ~)

Арифметическое отрицание (-) изменяет знак операнда целого или вещественного типа на противоположный

Логическое отрицание (!) дает в результате значение 0, если операнд есть истина (не ноль), и значение 1, если операнд равен нулю

Поразрядное отрицание (~), или побитовое, инвертирует каждый разряд в двоичном представлении целочисленного операнда

```
int a = 2, c;  
c = -a; //c = -2
```

```
int a = 2;  
bool t;  
t = !a; //t = false
```

```
short int a = 5, b;  
b = ~a; //b = -6
```


БИНАРНЫЕ ОПЕРАЦИИ

Сложение (+)
вычитание (-)
умножение (*)

Операции применимы
к операндам
арифметических типов

Ошибка!

```
int a;  
float b=9.9, c=1;  
a = b + c; //a=10
```

Верно:

```
int a=1;  
float b=9.6, c;  
c = a + b; //c=10.6
```

БИНАРНЫЕ ОПЕРАЦИИ

Деление (/) и остаток от деления (%)

Операция **деления** применима к операндам арифметических типов

Операция **остатка от деления** применяется только к целочисленным операндам. Знак результата зависит от реализации компилятора

```
int x = 11, y = 4, c;
```

```
float z = 4, a, b;
```

```
a = x / y;
```

```
b = x / z;
```

```
c = x % y;
```

Результат:

```
a = 2, b = 2.75, c = 3
```

БИНАРНЫЕ ОПЕРАЦИИ

Операции сдвига (<< и >>)

Операции применяются только к целочисленным операндам

Сдвигают двоичное представление первого операнда влево или вправо на количество двоичных разрядов, заданное вторым операндом

```
short int x = 11, y = 4, c;  
c = x << y; //c = 176
```

БИНАРНЫЕ ОПЕРАЦИИ

Операции отношения (<, <=, >, >=, ==, !=)

Операция сравнивают первый операнд со вторым

Результатом операции является значение **true** или **false**

БИНАРНЫЕ ОПЕРАЦИИ

Поразрядные операции (&, |, ^)

Операции применяются только к целочисленным операндам и работают с их двоичными представлениями

При поразрядной **конъюнкции** (&) бит результата равен 1 только тогда, когда соответствующие биты обоих операндов равны 1.

При поразрядной **дизъюнкции** (|) бит результата равен 1 тогда, когда соответствующий бит хотя бы одного из операндов равен 1.

При поразрядном **исключающем ИЛИ** (^) бит результата равен 1 только тогда, когда соответствующие биты операндов различны

```
short int x = 12, y = 5, c;  
c = x | y;  
cout << c; // c=13
```

БИНАРНЫЕ ОПЕРАЦИИ

Логические операции (&& и ||)

Операнды логических операций **И (&&)** и **ИЛИ (||)** могут иметь арифметический тип, при этом операнды в каждой операции могут быть различных типов

Преобразование типов **не** производится. Каждый операнд оценивается с точки зрения его эквивалентности нулю

```
short int x = 12;  
float y = 1.5;  
bool c;  
c = x && y; //c = true
```

БИНАРНЫЕ ОПЕРАЦИИ

Операции присваивания (=, +=, -=, *= и т. д.)

- В C++ можно использовать составные операторы присваивания
<переменная><знак_операции> = <выражение>;
Аналог: **<переменная> = <переменная><знак_операции><выражение>;**
b-=2; //b=b-2
c*=b+2; //c=c*(b+2);
- В C++ допустима запись:
<переменная 1> = <переменная 2> = ... <переменная N> =<выражение>;
int y=2,a,b,c;
a=b=c=2*y; //a=4, b=4, c=4

ТЕРНАРНАЯ (УСЛОВНАЯ) ОПЕРАЦИЯ

Операция имеет **три** операнда

<выражение 1>?<выражение 2>:<выражение 3>

Если результат вычисления выражения 1 равен true, то результатом условной операции будет результат второго выражения, иначе – третьего.

```
int a = 11, b = 4, maximum;  
maximum = (b > a) ? b*3 : a+2;  
cout << maximum;//13
```


ПРИОРИТЕТЫ ВЫПОЛНЕНИЯ ОПЕРАЦИЙ

Приоритет операций — это порядок выполнения операций в выражении, при условии, что в выражении не указан явный порядок выполнения операций (с использованием круглых скобок)

Ассоциативность — направление выполнения операций в случае, если операции имеют одинаковый приоритет (слева направо или справа налево)

приоритет	операция	ассоциативность	описание
1	()		круглые скобки
2	++, --	слева направо	постфиксный инкремент и декремент
3	++, --	справа налево	префиксный инкремент и декремент
4	*, /, %	слева направо	умножение, деление, остаток от деления
5	+, -	слева направо	сложение, вычитание
6	>>, <<	слева направо	побитовый сдвиг вправо и влево
7	<=, >, >=	слева направо	меньше, меньше равно, больше, больше равно
8	==, !=	слева направо	равно, не равно
9	&&	слева направо	логическое умножение
10	&	слева направо	поразрядное логическое умножение
11	^	слева направо	поразрядное исключающее ИЛИ
12		слева направо	поразрядное логическое сложение
13		слева направо	логическое сложение
14	?:	справа налево	тернарная операция
15	=, *=, /= ...	справа налево	все виды присваивания

функция	пояснение	пример
abs(x)	Модуль числа x	abs(-5.7)=5.7
asin(x)	Арксинус x, где $-1 < x < 1$	asin(0)=0
ceil(x)	Округление числа x до наименьшего целого, превосходящего x	ceil(12.1)=13 ceil(-9.8)=-9
sin(x)	Синус x (x задается в радианах)	sin(0)=0
cos(x)	Косинус x (x задается в радианах)	cos(0)=1
exp(x)	Экспонента числа x	exp(2.1)=8.16617
floor(x)	Округление x до наибольшего целого, не превосходящего x	floor(-3.7)=-4 floor(11.9)=11
fmod(x,y)	Остаток от деления x на y, где x и y - вещественные числа	fmod(7.6,2.3)=0.7 fmod(-6.7,-2.1)=-0.4
log(a)	Натуральный логарифм x	log(1)=0
log10(x)	Десятичный логарифм x	log10(100)=2
pow(x,y)	Возведение x в степень y	pow(3.5, 4)=150.063
sqrt(x)	Квадратный корень из x	sqrt(9.8)=3.1305

МАТЕМАТИЧЕСКИЕ КОНСТАНТЫ

```
#define _USE_MATH_DEFINES
```

```
# include <cmath>
```

M_E – экспонента

M_PI – число π

M_LOG2E – $\log_2(e)$

M_LOG10E – $\log_{10}(e)$

M_LN2 – $\ln(2)$

M_LN10 – $\ln(10)$

M_PI_2 – $\pi/2$

M_PI_4 – $\pi/4$

ОПЕРАТОР ВЕТВЛЕНИЯ

- Оператор ветвления позволяет выбрать один из двух вариантов продолжения вычислительного процесса в зависимости от истинности условия.

- Неполный вариант:

```
if (<условие>)  
{  
    блок инструкций;  
}
```

- В качестве условия может выступать любое выражение, дающее в результате логическое значение или выражение, которое можно свести к логическому значению

- Если условие дает значение «истина», то будет выполняться тело оператора, в противном случае никаких действий выполняться не будет

```
int x;  
cin >> x;  
if (x<0)  
    x=x*x;  
cout<< x;
```

ОПЕРАТОР ВЕТВЛЕНИЯ

- Полный вариант:

```
if (<условие>)  
{  
    блок инструкций_1;  
}  
else  
{  
    блок инструкций_2;  
}
```

- Если условие дает значение «истина», то будет выполняться блок инструкций_1, иначе – блок инструкций_2.

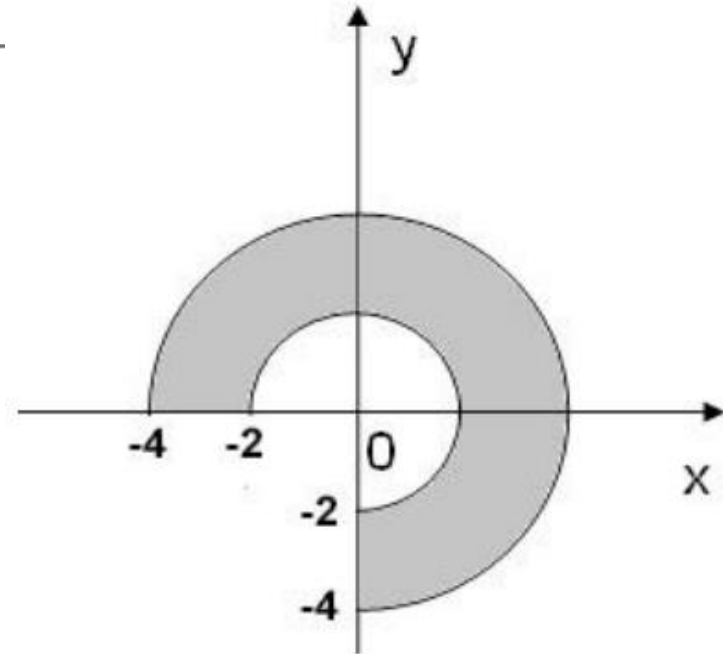
```
int x;  
cin >> x;  
if (x % 2 == 0)  
    cout<< "Число четное";  
else  
    cout<< "Число нечетное";
```

СЛОЖНЫЕ УСЛОВИЯ В ОПЕРАТОРЕ ВЕТВЛЕНИЯ

```
#include <iostream>
#include <cmath>
```

```
using namespace std;
```

```
void main() {
    float x, y;
    cout << "Введите координату x ";
    cin >> x;
    cout << "Введите координату y ";
    cin >> y;
    if ((x*x + y*y <= 16 && x*x + y*y >= 4) && !(x<0 && y<0))
        cout << "Точка принадлежит выделенной области";
    else
        cout << "Точка не принадлежит выделенной области";
}
```



ОПЕРАТОР ВЫБОРА

```
switch (<выражение>)  
{  
    case <константа_1>: <блок_1>; break;  
    case <константа_2>: <блок_2>; break;  
    ...  
    case <константа_n>: <блок_n>; break;  
    [default: <блок_по_умолчанию>;  
    break;]  
}
```

```
#include <iostream>  
using namespace std;  
void main () {  
    unsigned int D;  
    cout << "D=";  
    cin >> D;  
    switch (D%7)  
    {  
        case 1: cout<<"Monday"; break;  
        case 2: cout<<"Tuesday"; break;  
        case 3: cout<<"Wednesday"; break;  
        case 4: cout<<"Thursday"; break;  
        case 5: cout<<"Friday"; break;  
        case 6: cout<<"Saturday"; break;  
        case 0: cout<<"Sunday"; break;  
    }  
}
```

Вывести название дня недели, соответствующее заданному числу D, при условии, что в месяце 31 день и 1-е число — понедельник.

ГЕНЕРАТОР СЛУЧАЙНЫХ ЧИСЕЛ

```
#include<iostream>
#include<iomanip>
#include<cmath>
#include<ctime>
using namespace std;
void main()
{
    srand(time(0));
    cout << rand() << endl;
    cout << rand() << endl;
    cout << rand() << endl;
}
```

```
cout << rand() % 10; //целое число от 0 до 9
cout << rand() % 10 + 1; //целое число от 1 до 10
cout << rand() % 2 + 1; //целое число от 1 до 2
cout << rand() % 100 + 1; //целое число от 1 до 100
cout << rand() % 101; //целое число от 0 до 100
```

```
cout << (rand() % 101) - 50; //целое число от -50 до 50
```

ОПЕРАТОРЫ ПЕРЕБОРА В C++

Инструкция	Время вычисления	Инициализация	Приращение
for	начало цикла	+	+
range-based for	начало цикла	+	+
while	начало цикла	-	-
do	конец цикла	-	-

ОПЕРАТОР WHILE

```
while ( expression )  
    statement
```

выполняет инструкции
пока выражение не будет
иметь нулевое значение

```
while (x) {  
    if (x % 10 % 2 != 0)  
        count++;  
    x = x / 10;  
}
```

```
#include <iostream>  
using namespace std;  
void main() {  
    int count = 0, x;  
    cin >> x;  
    while (x != 0) {  
        if (x % 10 % 2 != 0)  
            count++;  
        x = x / 10;  
    }  
    cout << count;  
}
```

ВЫРАЖЕНИЕ DO-WHILE

```
do  
    statement  
while ( expression ) ;
```

```
#include <iostream>  
#define PASSWORD 1234  
using namespace std;  
void main()  
{  
    int x, i = 1;  
    do  
    {  
        cout << "Your password: ";  
        cin >> x;  
        i++;  
    } while (x != PASSWORD && i <= 3);  
    if (x == PASSWORD)  
        cout << "Success!";  
    else  
        cout << "Access Denied!";  
}
```

GCD

```
#include <iostream>
using namespace std;
- void main()
{
    int n, m;
    cin >> n >> m;
    while (n != 0 && m != 0)
        if (m >= n) m %= n;
        else n %= m;
    cout << n + m;
}
```

$$GSD(M, N) = \begin{cases} M + N, & \text{if } M = 0 \text{ or } N = 0 \\ GSD(M \bmod N, N), & \text{if } M \geq N \\ GSD(M, N \bmod M), & \text{if } N > M \end{cases}$$

ОПРЕДЕЛЕНИЕ ПРОСТОТЫ ЧИСЛА

```
#include <iostream>
using namespace std;
void main()
{
    int N, i = 2;
    cin >> N;
    if (N == 2) cout << "Prime";
    else
    {
        if (N % 2 == 0)
            cout << "Composite";
        else
        {
            do
                i++;
            while (i * i <= N && N % i != 0);
            if (i * i <= N) cout << "Composite";
            else cout << "Prime";
        }
    }
}
```

ФАКТОРИЗАЦИЯ ЧИСЛА

```
while (k % i != 0)
    if (i * i > N) i = k;
    else i += 1;
```

```
#include <iostream>
using namespace std;
void main()
{
    int N, k, i;
    cin >> N;
    k = N;
    while (k != 1)
    {
        i = 2;
        while (k % i != 0)
            i += 1;
        cout << i << " ";
        k /= i;
    }
}
```

ВЫЧИСЛЕНИЕ СУММЫ РЯДА С ЗАДАННОЙ ТОЧНОСТЬЮ

$$S = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} + \dots$$

$$\varepsilon = 0.001$$

```
#include <iostream>
#include <cmath>
using namespace std;
void main()
{
    int i = 1;
    float s = 0, eps = 0.001, a = 1;
    while (abs(a) > eps)
    {
        s = s + a;
        i++;
        a = 1 / float(i);
    }
    cout << s;
}
```


ОПЕРАТОР FOR

```
for (init - expression; cond - expression; loop - expression)
    statement
```

```
for (int i = 0; i < 10; i++) {
    :
    cout << i;
}
```

```
int i;
for (i = 0; i < 10; i++) {
    :
    cout << i;
}
```

количество проверок условия цикла на 1 больше, чем количество итераций цикла

может быть ни разу не выполнен, если условие выполнения цикла бу

```
for (i = 1; i < 1; i--)
```

может произойти заикливание, если условие никогда не станет ложным

```
for (i = 10; i <= 10; i--)
```

```
#include <iostream>
using namespace std;
```

```
void main() {
    int N, S;
    cin >> N;
    S = 0;
    for (int i = 1; i <= N; i++)
        S = S + i;
    cout << S;
}
```

Вычислить сумму ряда от 1 до N

итерация	i	S	i<=N
0	-	0	
1	1	1	true
2	2	3	true
3	3	6	true
4	4	10	true
5	5	15	true
6	6		false

Вычислить произведение
 $2*4*6*8*...*2N$

```
#include <iostream>
using namespace std;
void main() {
    int i, a, N, P;
    cin >> N;
    for (i = 1, P = 1, a = 2; i <= N; i++, a = a + 2)
        P = P * a;
    cout << P;
}
```

```
#include <iostream>
using namespace std;
void main() {
    int i, N, P;
    cin >> N;
    for (i = 1, P = 1; i <= N; i++)
        P = P * (2 * i);
    cout << P;
}
```

ПЛОХОЙ ПРИМЕР

Вычислить произведение
 $1*2*3*4*...*N$

```
for (;;)
    <тело цикла>
```

```
#include <iostream>
using namespace std;
void main() {
    int N, i, P;
    cin >> N;
    i = 1;
    P = 1;
    for (;;) {
        P = P * i;
        i = i + 1;
        if (i > N) break;
    }
    cout << P;
}
```

ОПЕРАТОР ПЕРЕХОДА BREAK

`break;`

предназначен для
искусственного прерывания
выполнения

- последовательности итераций в операторах цикла for, while или do-while
- последовательности операторов в операторе выбора switch

«вечный»
цикл

```
#include <iostream>
using namespace std;
```

```
int main()
{
    int number, summ = 0;
    for (;;)
    {
        cin >> number;
        if (number == 0)
            break;
        summ += number;
    }
    cout << summ;
    return 0;
}
```

```
for (i = 0; i < 5; i++)
{
    if (i == 3)
        break;
    cout << "i = " << i << endl;
}
```

```
i = 0;
while (i < 5)
{
    if (i == 3)
        break;
    cout << "i = " << i << endl;
    i++;
}
```

```
i = 0;
do
{
    if (i == 3)
        break;
    cout << "i = " << i << endl;
    i++;
} while (i < 5);
```

i	=	0
i	=	1
i	=	2

ОПЕРАТОР ПЕРЕХОДА CONTINUE

`continue;`

Предназначен для перехода к выполнению следующей итерации цикла. Если в теле цикла встречается оператор `continue`:

- выполнение текущей итерации останавливается
- происходит переход к следующей итерации цикла

```
#include <iostream>
using namespace std;

int main()
{
    for (int i = 1; i <= 100; i++)
    {
        if (i % 5 != 0)
        {
            continue;
        }
        cout << i << endl;
    }
    return 0;
}
```

```
for (i = 0; i < 5; i++)
{
    if (i == 3)
        continue;
    cout << "i = " << i << endl;
}
```

```
i = 0;
while (i < 5)
{
    if (i == 3)
    {
        i++;
        continue;
    }
    cout << "i = " << i << endl;
    i++;
}
```

```
i = 0;
do
{
    if (i == 3)
    {
        i++;
        continue;
    }
    cout << "i = " << i << endl;
    i++;
} while (i < 5);
```

```
i = 0
i = 1
i = 2
i = 4
```


ВЛОЖЕННЫЕ ЦИКЛЫ

```
#include <iostream>
using namespace std;
void main() {
    for (int i = 1; i <= 10; i++) {
        for (int j = 1; j <= 10; j++) {
            cout << i << " * " << j << " = " << i * j;
            cout << endl;
        }
        cout << endl;
    }
    cout << endl;
}
```

ОСНОВАННОЕ НА ДИАПАЗОНЕ ВЫРАЖЕНИЕ FOR

```
#include<iostream>
using namespace std;
int main()
{
    int arr[] = { 1,2,3,4,5 };
    cout << "The elements are: ";
    for (int i : arr)
    {
        cout << i << " ";
    }
    return 0;
}
```

for (for-range-declaration:expression)
statement

```
for (auto var : arr)
{
    cout << var << " ";
}
```

